

Submitted by: Ashvin Bhatt

Week: 1

Topic: Python Fundamentals for Data Analytics

1. Task 1 – Python Basics

Objective

To understand basic Python syntax and practice taking user input and displaying output.

Brief Explanation

This task introduced the basic structure of a Python program. The program displayed a welcome message and collected basic information from the user.

Python Source Code

```
# Welcome message

print("Welcome to the Python program!\n")

# Take user input

name = input("Please enter your name: ")

college = input("Please enter your college name: ")

branch = input("Please enter your branch: ")

# Displaying information

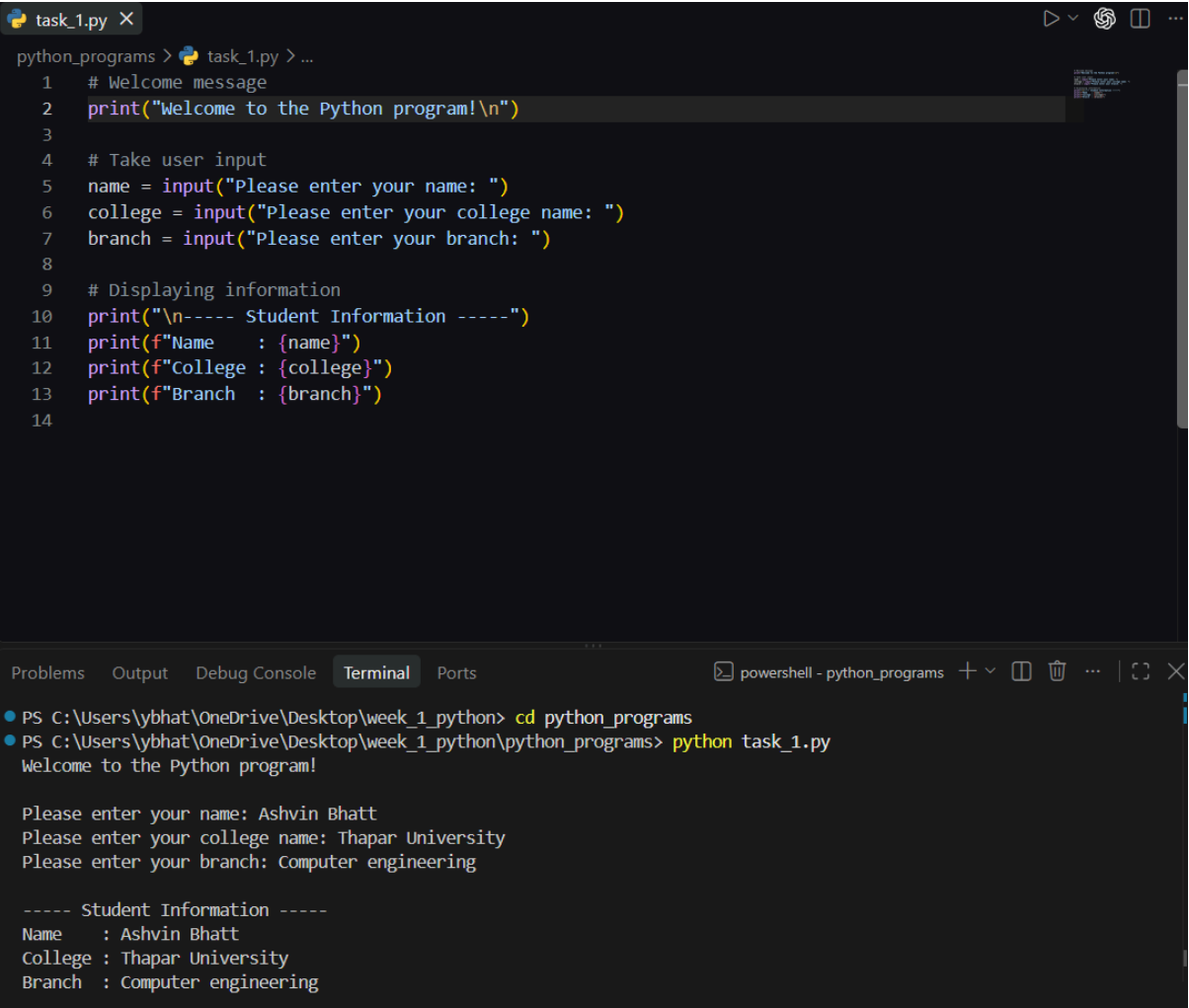
print("\n----- Student Information -----")

print(f"Name      : {name}")

print(f"College   : {college}")

print(f"Branch    : {branch}")
```

Screenshot of Program Output



The screenshot shows a code editor window with a file named `task_1.py`. The code is a Python script that takes user input for name, college, and branch, and then displays the information in a formatted output.

```
python_programs > task_1.py > ...
1  # Welcome message
2  print("welcome to the Python program!\n")
3
4  # Take user input
5  name = input("Please enter your name: ")
6  college = input("Please enter your college name: ")
7  branch = input("Please enter your branch: ")
8
9  # Displaying information
10 print("\n----- Student Information -----")
11 print(f"Name      : {name}")
12 print(f"College   : {college}")
13 print(f"Branch    : {branch}")
14
```

Below the code editor, the terminal window shows the execution of the program. The user has navigated to the `python_programs` directory and run `python task_1.py`. The output shows the program's execution, including the welcome message and the user's input.

```
PS C:\Users\ybhat\OneDrive\Desktop\week_1_python> cd python_programs
PS C:\Users\ybhat\OneDrive\Desktop\week_1_python\python_programs> python task_1.py
Welcome to the Python program!

Please enter your name: Ashvin Bhatt
Please enter your college name: Thapar University
Please enter your branch: Computer engineering

----- Student Information -----
Name      : Ashvin Bhatt
College   : Thapar University
Branch    : Computer engineering
```

2. Task 2 – Variables and Data Types

Objective

To learn how to create variables and work with Python data types.

Brief Explanation

This task demonstrated how to store values in variables and identify or use common Python data types such as strings, integers, floats, and booleans.

Python Source Code

```
# Declaring variables

age = 22                                #integer
percentage = 85.5                       #float
name = "Ashvin"                         #string
is_student = True                       #boolean

# Displaying values and their data types

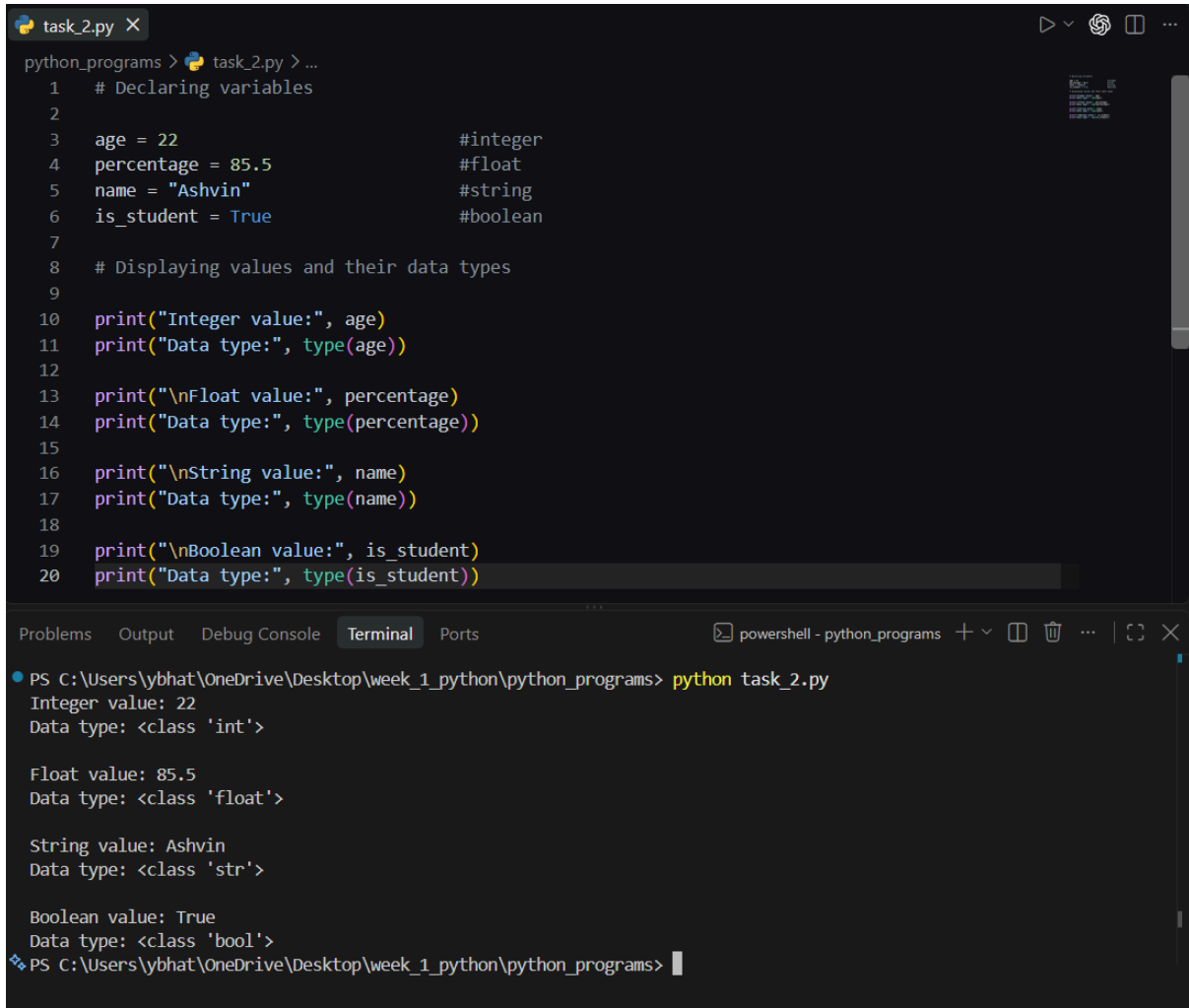
print("Integer value:", age)
print("Data type:", type(age))

print("\nFloat value:", percentage)
print("Data type:", type(percenta

print("\nString value:", name)
print("Data type:", type(name))

print("\nBoolean value:", is_student)
print("Data type:", type(is_student))
```

Screenshot of Program Output



The screenshot shows a code editor with a file named `task_2.py`. The code declares four variables: `age` (integer), `percentage` (float), `name` (string), and `is_student` (boolean). It then prints each variable's value and its data type. Below the code, a terminal window shows the output of running `python task_2.py`. The output displays the values and their corresponding data types: Integer value: 22, Data type: <class 'int'>, Float value: 85.5, Data type: <class 'float'>, String value: Ashvin, Data type: <class 'str'>, and Boolean value: True, Data type: <class 'bool'>.

```
python_programs > task_2.py > ...
1  # Declaring variables
2
3  age = 22                                #integer
4  percentage = 85.5                       #float
5  name = "Ashvin"                         #string
6  is_student = True                       #boolean
7
8  # Displaying values and their data types
9
10 print("Integer value:", age)
11 print("Data type:", type(age))
12
13 print("\nFloat value:", percentage)
14 print("Data type:", type(percentage))
15
16 print("\nString value:", name)
17 print("Data type:", type(name))
18
19 print("\nBoolean value:", is_student)
20 print("Data type:", type(is_student))
```

```
PS C:\Users\yhat\OneDrive\Desktop\week_1_python\python_programs> python task_2.py
Integer value: 22
Data type: <class 'int'>

Float value: 85.5
Data type: <class 'float'>

String value: Ashvin
Data type: <class 'str'>

Boolean value: True
Data type: <class 'bool'>
PS C:\Users\yhat\OneDrive\Desktop\week_1_python\python_programs>
```

3. Task 3 – Operators

Objective

To perform arithmetic operations using Python.

Brief Explanation

This task used user input and arithmetic operators to perform calculations such as addition, subtraction, multiplication, and division and modulus.

Python Source Code

```
# input

num1 = float(input("Enter first number: "))

num2 = float(input("Enter second number: "))

# operations

sum = num1 + num2

difference = num1 - num2

product = num1 * num2

# results

print(f"Sum: {sum}")

print(f"Difference: {difference}")

print(f"Product: {product}")

# division and modulus

if num2 != 0:

    division = num1 / num2

    modulus = num1 % num2

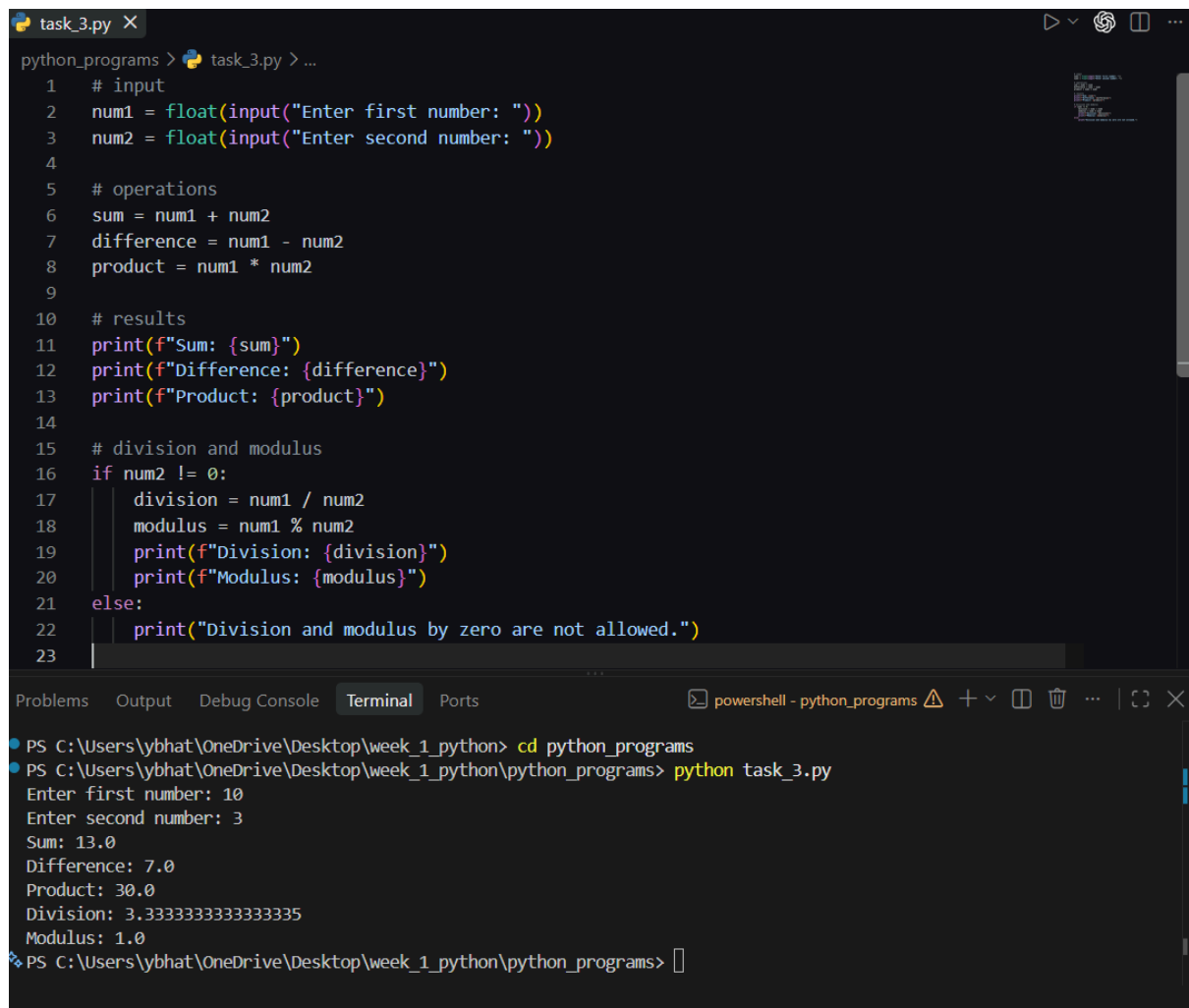
    print(f"Division: {division}")

    print(f"Modulus: {modulus}")

else:

    print("Division and modulus by zero are not allowed.")
```

Screenshot of Program Output



The screenshot displays a code editor window titled 'task_3.py' with a dark theme. The Python code defines two input numbers, calculates their sum, difference, product, division, and modulus, and prints the results. A conditional statement checks for division by zero. Below the code, the 'Terminal' tab shows the execution of the program, including user input and the resulting calculations.

```
python_programs > task_3.py > ...
1  # input
2  num1 = float(input("Enter first number: "))
3  num2 = float(input("Enter second number: "))
4
5  # operations
6  sum = num1 + num2
7  difference = num1 - num2
8  product = num1 * num2
9
10 # results
11 print(f"Sum: {sum}")
12 print(f"Difference: {difference}")
13 print(f"Product: {product}")
14
15 # division and modulus
16 if num2 != 0:
17     division = num1 / num2
18     modulus = num1 % num2
19     print(f"Division: {division}")
20     print(f"Modulus: {modulus}")
21 else:
22     print("Division and modulus by zero are not allowed.")
23
```

Terminal Output:

```
PS C:\Users\ybhat\OneDrive\Desktop\week_1_python> cd python_programs
PS C:\Users\ybhat\OneDrive\Desktop\week_1_python\python_programs> python task_3.py
Enter first number: 10
Enter second number: 3
Sum: 13.0
Difference: 7.0
Product: 30.0
Division: 3.3333333333333335
Modulus: 1.0
PS C:\Users\ybhat\OneDrive\Desktop\week_1_python\python_programs>
```

4. Task 4 – Conditional Statements

Objective

To calculate a grade based on marks or a score.

Brief Explanation

This task used conditional statements to evaluate a score and display the corresponding grade.

Python Source Code

```
# input marks

marks = float(input("Enter your marks (0-100): "))

# Checking marks and displaying the grade

if marks < 0 or marks > 100:

    print("Invalid marks! Please enter marks between 0 and 100.")

elif marks >= 90:

    print("Grade: A")

elif marks >= 75:

    print("Grade: B")

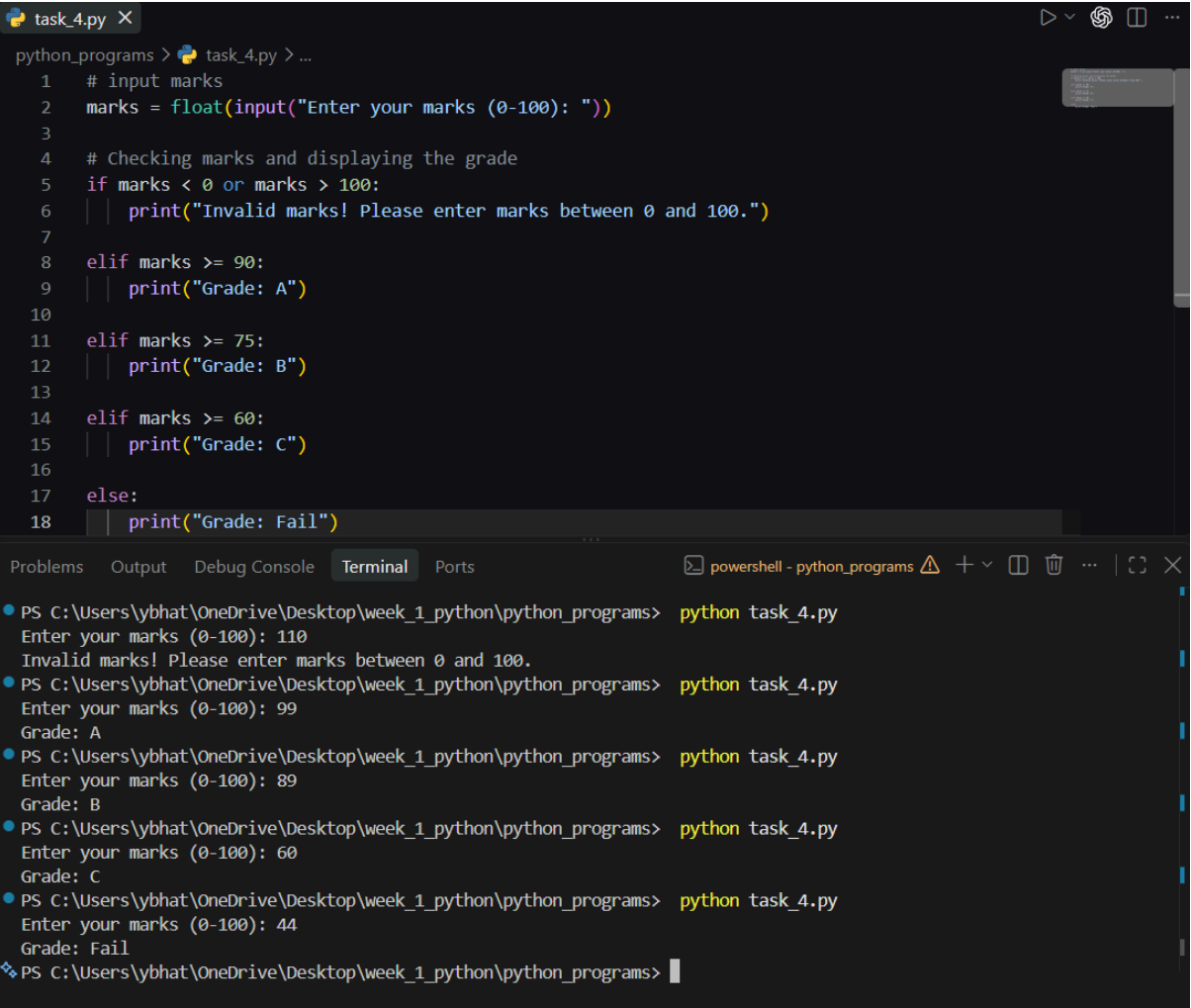
elif marks >= 60:

    print("Grade: C")

else:

    print("Grade: Fail")
```

Screenshot of Program Output



The screenshot displays a code editor window titled 'task_4.py' and a terminal window below it. The code in the editor is a Python script that takes a mark as input and prints a corresponding grade based on a set of conditions. The terminal shows the output of running this script five times with different input values.

```
python_programs > task_4.py > ...
1  # input marks
2  marks = float(input("Enter your marks (0-100): "))
3
4  # Checking marks and displaying the grade
5  if marks < 0 or marks > 100:
6      print("Invalid marks! Please enter marks between 0 and 100.")
7
8  elif marks >= 90:
9      print("Grade: A")
10
11 elif marks >= 75:
12     print("Grade: B")
13
14 elif marks >= 60:
15     print("Grade: C")
16
17 else:
18     print("Grade: Fail")
```

Terminal Output:

```
PS C:\Users\ybhat\OneDrive\Desktop\week_1_python\python_programs> python task_4.py
Enter your marks (0-100): 110
Invalid marks! Please enter marks between 0 and 100.
PS C:\Users\ybhat\OneDrive\Desktop\week_1_python\python_programs> python task_4.py
Enter your marks (0-100): 99
Grade: A
PS C:\Users\ybhat\OneDrive\Desktop\week_1_python\python_programs> python task_4.py
Enter your marks (0-100): 89
Grade: B
PS C:\Users\ybhat\OneDrive\Desktop\week_1_python\python_programs> python task_4.py
Enter your marks (0-100): 60
Grade: C
PS C:\Users\ybhat\OneDrive\Desktop\week_1_python\python_programs> python task_4.py
Enter your marks (0-100): 44
Grade: Fail
PS C:\Users\ybhat\OneDrive\Desktop\week_1_python\python_programs>
```

5. Task 5 – Loops

Objective

To understand repetition using Python loops.

Brief Explanation

This task practiced using loops to execute a block of code repeatedly, helping develop an understanding of iteration.

Python Source Code

```
# Program 1: Print numbers from 1 to 20 using for loop

print("Numbers from 1 to 20:")

for number in range(1, 21):

    print(number, end=" ")

print("\n")

# Program 2: Multiplication table

num = int(input("Enter a number for multiplication table: "))

for i in range(1, 11):

    print(f"{num} x {i} = {num * i}")

# Program 3: Print even numbers from 1 to 50 using while loop

print("\nEven numbers from 1 to 50:")

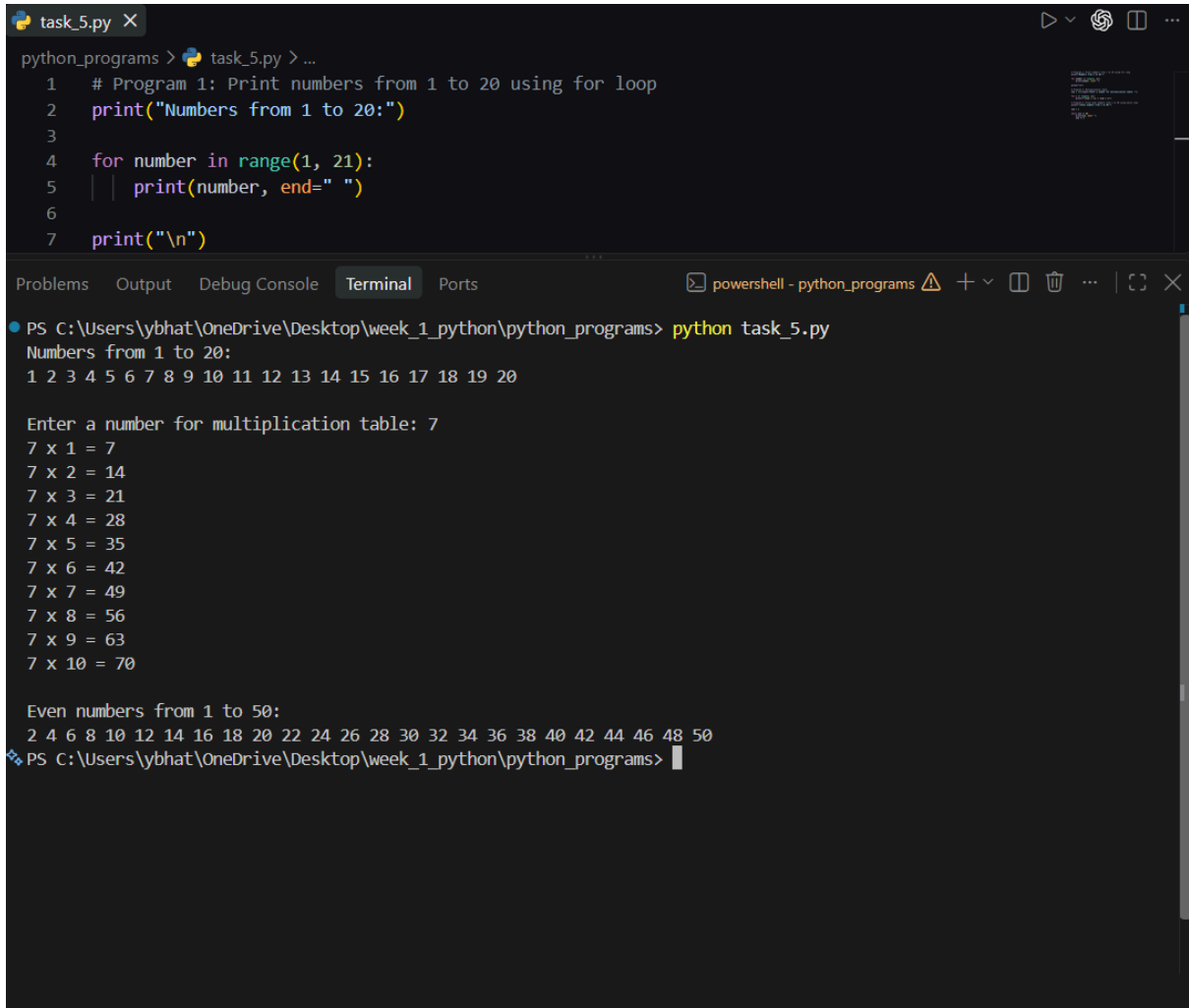
num = 2

while num <= 50:

    print(num, end=" ")

    num += 2
```

Screenshot of Program Output



The screenshot shows a Visual Studio Code editor with a file named `task_5.py` open. The code in the editor is as follows:

```
python_programs > task_5.py > ...
1  # Program 1: Print numbers from 1 to 20 using for loop
2  print("Numbers from 1 to 20:")
3
4  for number in range(1, 21):
5      print(number, end=" ")
6
7  print("\n")
```

The terminal window at the bottom shows the execution of the script. The output is as follows:

```
PS C:\Users\ybhat\OneDrive\Desktop\week_1_python\python_programs> python task_5.py
Numbers from 1 to 20:
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20

Enter a number for multiplication table: 7
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70

Even numbers from 1 to 50:
2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50
PS C:\Users\ybhat\OneDrive\Desktop\week_1_python\python_programs>
```

6. Task 6 – Functions

Objective

To learn how to define and call reusable functions.

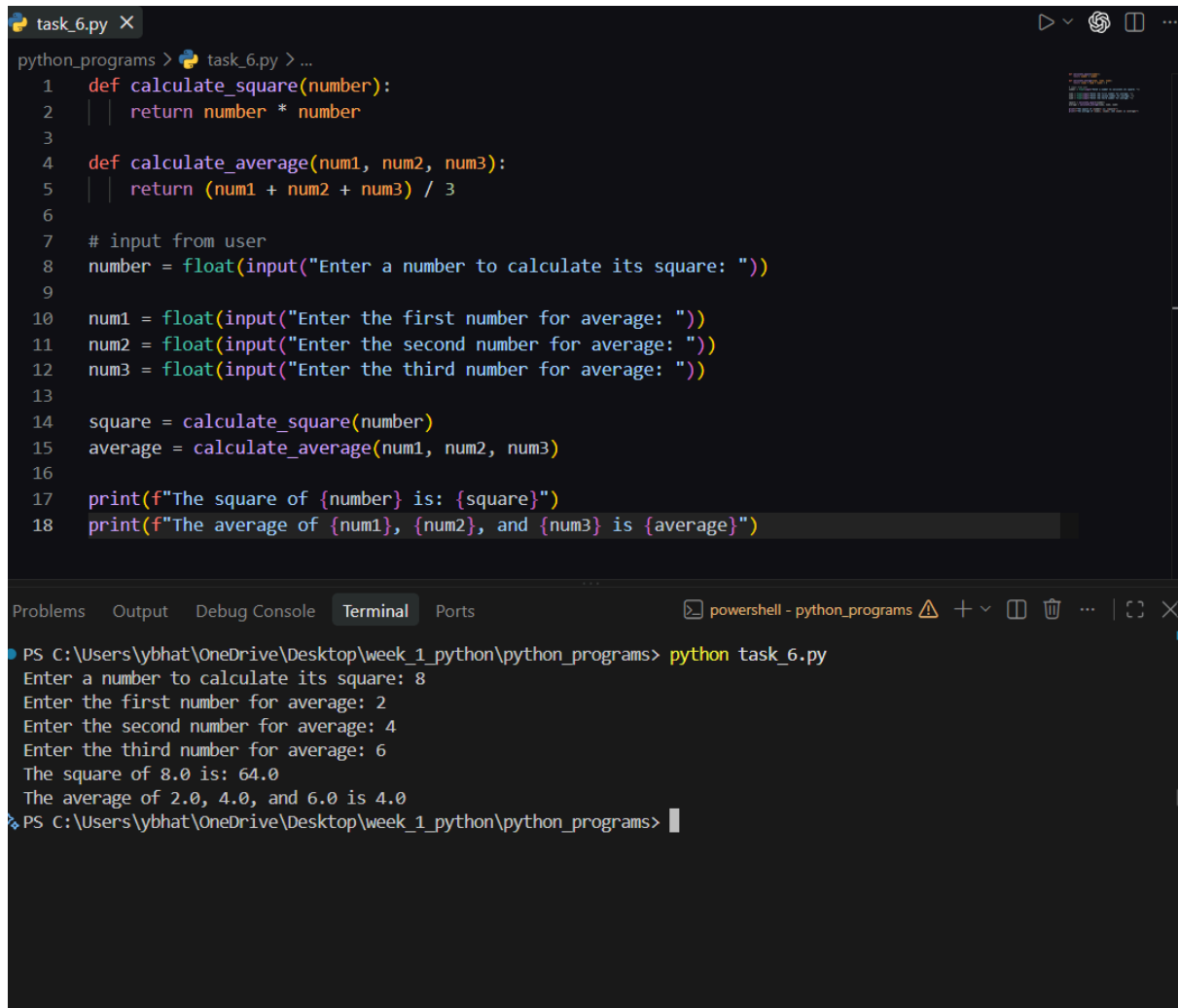
Brief Explanation

This task demonstrated how functions organize code into reusable blocks and how parameters and return values can be used.

Python Source Code

```
def calculate_square(number):  
    return number * number  
  
def calculate_average(num1, num2, num3):  
    return (num1 + num2 + num3) / 3  
  
# input from user  
  
number = float(input("Enter a number to calculate its square: "))  
  
num1 = float(input("Enter the first number for average: "))  
  
num2 = float(input("Enter the second number for average: "))  
  
num3 = float(input("Enter the third number for average: "))  
  
square = calculate_square(number)  
  
average = calculate_average(num1, num2, num3)  
  
print(f"The square of {number} is: {square}")  
  
print(f"The average of {num1}, {num2}, and {num3} is: {average}")
```

Screenshot of Program Output



```
task_6.py X
python_programs > task_6.py > ...
1  def calculate_square(number):
2      return number * number
3
4  def calculate_average(num1, num2, num3):
5      return (num1 + num2 + num3) / 3
6
7  # input from user
8  number = float(input("Enter a number to calculate its square: "))
9
10 num1 = float(input("Enter the first number for average: "))
11 num2 = float(input("Enter the second number for average: "))
12 num3 = float(input("Enter the third number for average: "))
13
14 square = calculate_square(number)
15 average = calculate_average(num1, num2, num3)
16
17 print(f"The square of {number} is: {square}")
18 print(f"The average of {num1}, {num2}, and {num3} is {average}")

Problems Output Debug Console Terminal Ports
powershell - python_programs
PS C:\Users\ybhat\OneDrive\Desktop\week_1_python\python_programs> python task_6.py
Enter a number to calculate its square: 8
Enter the first number for average: 2
Enter the second number for average: 4
Enter the third number for average: 6
The square of 8.0 is: 64.0
The average of 2.0, 4.0, and 6.0 is 4.0
PS C:\Users\ybhat\OneDrive\Desktop\week_1_python\python_programs>
```

7. Task 7 – Strings and Collections

Objective

To practice working with strings and Python collections.

Brief Explanation

This task explored string operations and collections such as lists, including accessing, modifying, and sorting data.

Python Source Code

```
# 1. String Operations

text = "Hello, World!"

print("String Operations")

print("Original String:", text)

print("Uppercase:", text.upper())

print("Lowercase:", text.lower())

print("Replace:", text.replace("World", "Duniya"))

print("Find 'World':", text.find("World"))

# 2. List Operations

fruits = ["apple", "banana", "cherry"]

print("\nList Operations")

print("Original List:", fruits)

fruits.append("mango")

print("After Append:", fruits)

fruits.remove("banana")

print("After Remove:", fruits)

fruits.sort()

print("After Sort:", fruits)

# 3. Tuple Creation and Indexing

student = ("Ashvin", 22, "Computer Engineering")

print("\nTuple Operations")

print("Student Tuple:", student)

print("Student Name:", student[0])

print("Student Age:", student[1])
```

```
print("Student Branch:", student[2])

# 4. Dictionary Operations

print("\nDictionary Operations")

student_info = {

    "name": "Ashvin",

    "college": "Thapar University",

    "branch": "Computer Engineering",

    "year": 2026

}

print("\nStudent Information:", student_info)

print("Student Name:", student_info["name"])

print("College:", student_info["college"])

print("Branch:", student_info["branch"])

print("Year:", student_info["year"])

# 5. Set Operations

print("\nSet Operations")

numbers = {10, 20, 30, 40}

print("Original Set:", numbers)

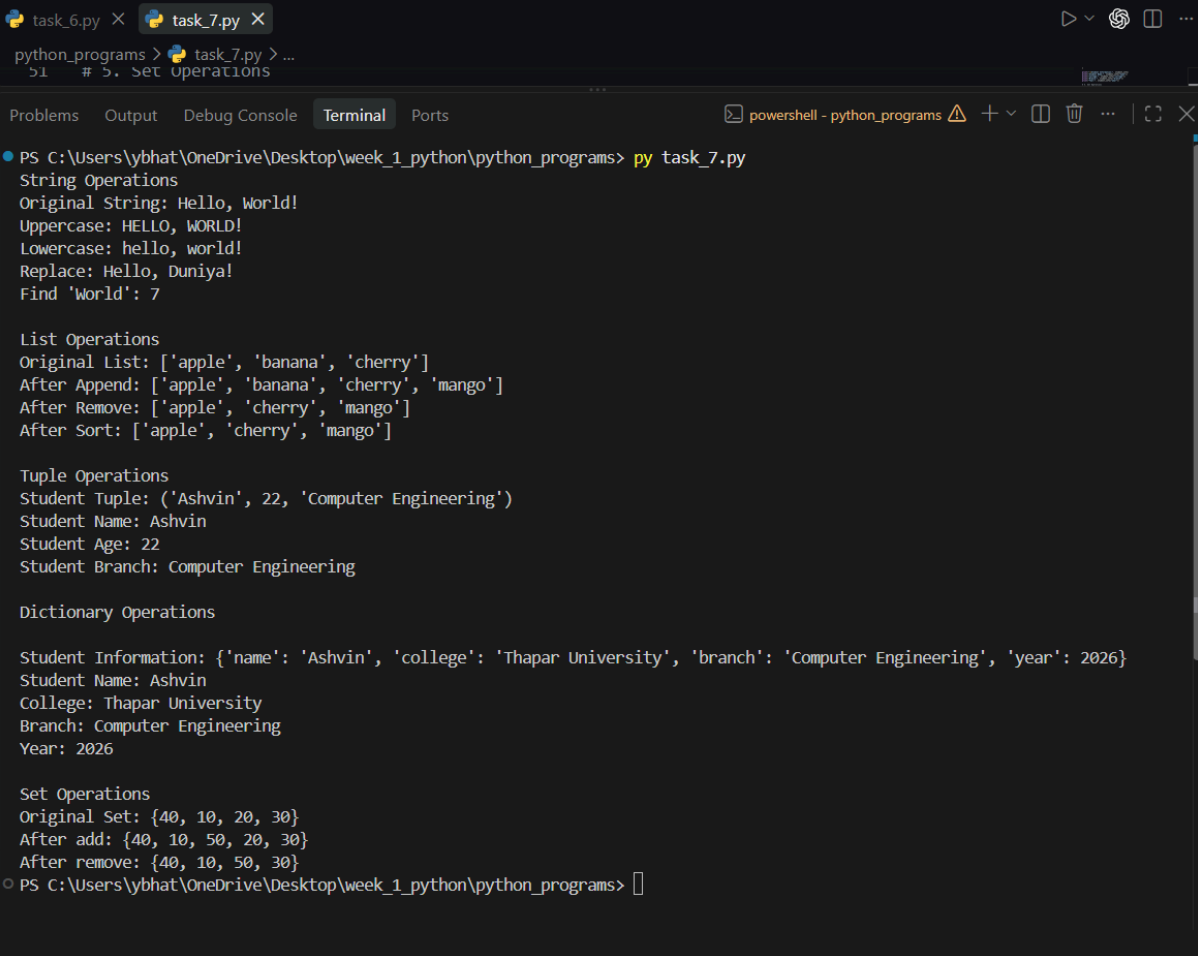
numbers.add(50)

print("After add:", numbers)

numbers.remove(20)

print("After remove:", numbers)
```

Screenshot of Program Output



The screenshot shows a terminal window with the following output:

```
task_6.py X task_7.py X
python_programs > task_7.py > ...
51 # 5. Set operations

PS C:\Users\ybhat\OneDrive\Desktop\week_1_python\python_programs> py task_7.py

String Operations
Original String: Hello, World!
Uppercase: HELLO, WORLD!
Lowercase: hello, world!
Replace: Hello, Duniya!
Find 'World': 7

List Operations
Original List: ['apple', 'banana', 'cherry']
After Append: ['apple', 'banana', 'cherry', 'mango']
After Remove: ['apple', 'cherry', 'mango']
After Sort: ['apple', 'cherry', 'mango']

Tuple Operations
Student Tuple: ('Ashvin', 22, 'Computer Engineering')
Student Name: Ashvin
Student Age: 22
Student Branch: Computer Engineering

Dictionary Operations

Student Information: {'name': 'Ashvin', 'college': 'Thapar University', 'branch': 'Computer Engineering', 'year': 2026}
Student Name: Ashvin
College: Thapar University
Branch: Computer Engineering
Year: 2026

Set Operations
Original Set: {40, 10, 20, 30}
After add: {40, 10, 50, 20, 30}
After remove: {40, 10, 50, 30}
PS C:\Users\ybhat\OneDrive\Desktop\week_1_python\python_programs> 
```

8. Task 8 – Basic File Handling

Objective

To read from or write to files using Python.

Brief Explanation

This task demonstrated file handling by working with a text file. It provided practice with opening files, reading or writing content, and handling file paths.

Python Source Code

```
name = "Ashvin Bhatt"

college = "Thapar University"

branch = "Computer Engineering"

introduction = (

    f"Hello, my name is {name}.\n"

    f"I study at {college}.\n"

    f"My branch is {branch}.\n"

    "I am learning Python for Data Analytics."

)

file_path = "introduction.txt"

with open(file_path, "w") as file:

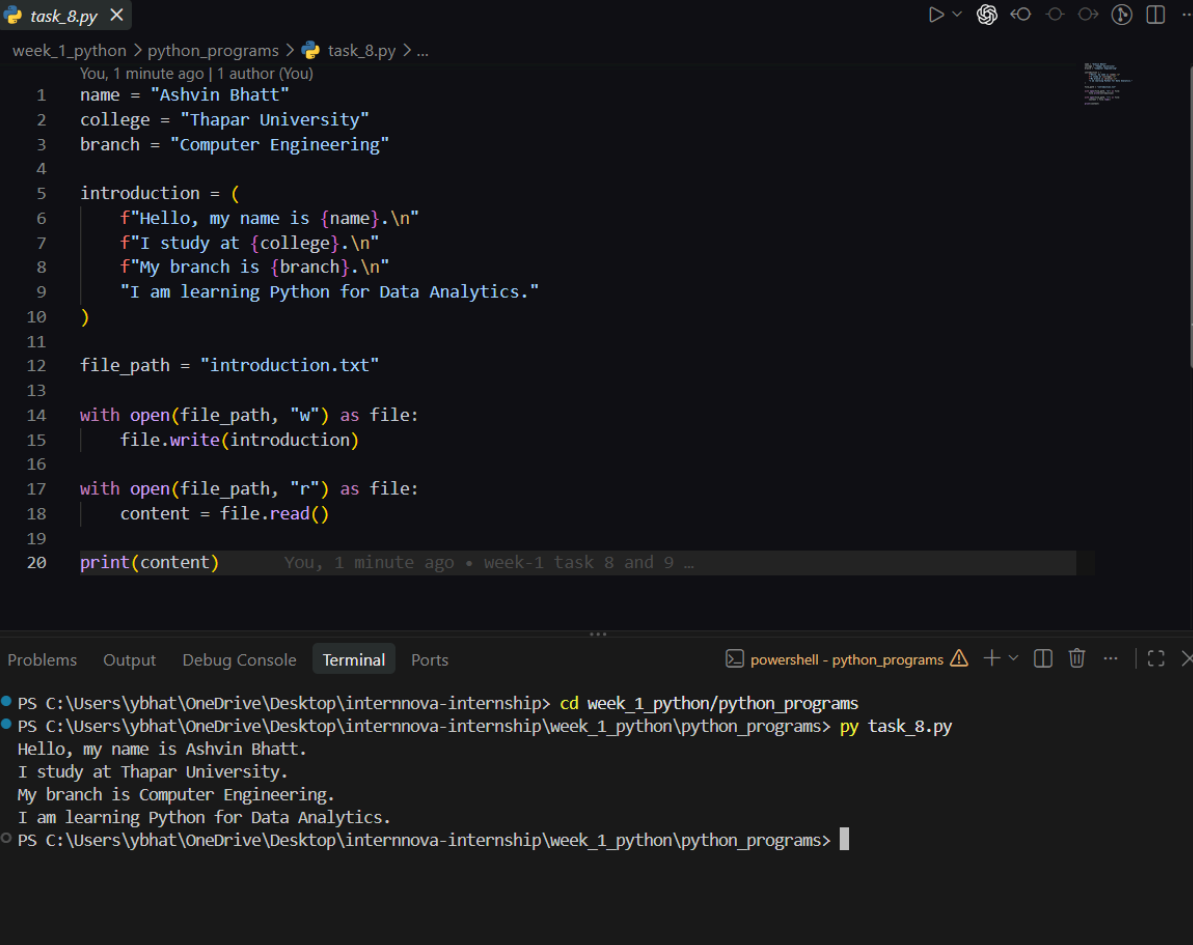
    file.write(introduction)

with open(file_path, "r") as file:

    content = file.read()

print(content)
```


Screenshot of Program Output



The screenshot displays a code editor window titled 'task_8.py'. The code defines variables for a student's name, college, and branch, then constructs a string introduction. It writes this string to a file named 'introduction.txt' and subsequently reads the file's content back to print it. Below the code, the terminal window shows the execution of the script, displaying the same introduction text that was written to the file.

```
task_8.py
week_1_python > python_programs > task_8.py > ...
You, 1 minute ago | 1 author (You)
1 name = "Ashvin Bhatt"
2 college = "Thapar University"
3 branch = "Computer Engineering"
4
5 introduction = (
6     f"Hello, my name is {name}.\n"
7     f"I study at {college}.\n"
8     f"My branch is {branch}.\n"
9     "I am learning Python for Data Analytics."
10 )
11
12 file_path = "introduction.txt"
13
14 with open(file_path, "w") as file:
15     file.write(introduction)
16
17 with open(file_path, "r") as file:
18     content = file.read()
19
20 print(content)
```

Terminal Output:

```
PS C:\Users\ybhat\OneDrive\Desktop\internnova-internship> cd week_1_python/python_programs
PS C:\Users\ybhat\OneDrive\Desktop\internnova-internship\week_1_python\python_programs> py task_8.py
Hello, my name is Ashvin Bhatt.
I study at Thapar University.
My branch is Computer Engineering.
I am learning Python for Data Analytics.
PS C:\Users\ybhat\OneDrive\Desktop\internnova-internship\week_1_python\python_programs>
```

9. Task 9 – Mini Python Project

Objective

To build a student record management system using python.

Brief Explanation

This task combined Python concepts such as lists, dictionaries, functions, loops, and conditional statements. The program supports adding, displaying, searching, and deleting student records.

Python Source Code

```
# list

students = []

# function to add student

def add_student():

    print("\n---Add Student---")

    name = input("Enter student name: ")

    roll_number = input("Enter roll number: ")

    branch = input("Enter branch: ")

    age = int(input("Enter age: "))

    student = {

        "name": name,

        "roll_number": roll_number,

        "branch": branch,

        "age": age

    }

    students.append(student)

    print("Student record added successfully!")


# function to display all students

def display_students():

    print("\n---All Students---")

    if not students:

        print("No student records found.")

    return
```

```

for student in students:

    print(f"Name: {student['name']}")

    print(f"Roll Number: {student['roll_number']}")

    print(f"Branch: {student['branch']}")

    print(f"Age: {student['age']}")

# function to search for a student by name
def search_student():

    print("\n---Search Student---")

    search_name = input("Enter student name to search: ").strip()

    found = False

    for student in students:

        if student["name"].lower() == search_name.lower():

            print("\nStudent Found:")

            print(f"Name: {student['name']}")

            print(f"Roll Number: {student['roll_number']}")

            print(f"Branch: {student['branch']}")

            print(f"Age: {student['age']}")

            found = True

    if not found:

        print("Student not found")

# function to delete a student by roll number
def delete_student():

```

```
print("\n---Delete Student---")

roll_number = input("Enter roll number to delete: ").strip()

for student in students:

    if student["roll_number"] == roll_number:

        students.remove(student)

        print("Student record deleted successfully!")

        return

print("Student record not found")

while True:

    print("\n---Student Record Management System---")

    print("1. Add Student")

    print("2. Display All Students")

    print("3. Search Student by Name")

    print("4. Delete Student")

    print("5. Exit")

    choice = input("Enter your choice (1-5): ")

    if choice == "1":

        add_student()

    elif choice == "2":

        display_students()

    elif choice == "3":

        search_student()

    elif choice == "4":

        delete_student()
```

```

elif choice == "5":

    print("Exiting Student Record Management System. Goodbye!")

    break

else:

    print("Invalid choice! Please enter a number from 1 to 5.")

```

Screenshot of Program Output

```

task_9.py U X
week_1_python > python_programs > task_9.py > add_student
5 def add_student():
...

PS C:\Users\ybhat\OneDrive\Desktop\internnova-internship\week_1_python\python_programs> py task_9.py
---Student Record Management System---
1. Add Student
2. Display All Students
3. Search Student by Name
4. Delete Student
5. Exit
Enter your choice (1-5): 1

---Add Student---
Enter student name: Yash
Enter roll number: 102213013
Enter branch: Civil Engineering
Enter age: 21
Student record added successfully!

---Student Record Management System---
1. Add Student
2. Display All Students
3. Search Student by Name
4. Delete Student
5. Exit
Enter your choice (1-5): 2

---All Students---
Name: Yash
Roll Number: 102213013
Branch: Civil Engineering
Age: 21

---Student Record Management System---
1. Add Student
2. Display All Students
3. Search Student by Name
4. Delete Student

```

Ln 6, Col 5 Spaces: 2 UTF-8 CRLF { } Python Python 3.8

```
task_9.py U X
week_1_python > python_programs > task_9.py > add_student
5 def add_student():
...

Problems Output Debug Console Terminal Ports
py - python_programs ⚠ + - [] ... | 🔍 ✕

PS C:\Users\ybbat\OneDrive\Desktop\internnova-internship\week_1_python\python_programs> py task_9.py
5. Exit
Enter your choice (1-5): 3

---Search Student---
Enter student name to search: Yash

Student Found:
Name: Yash
Roll Number: 102213013
Branch: Civil Engineering
Age: 21

---Student Record Management System---
1. Add Student
2. Display All Students
3. Search Student by Name
4. Delete Student
5. Exit
Enter your choice (1-5): 4

---Delete Student---
Enter roll number to delete: 102213013
Student record deleted successfully!

---Student Record Management System---
1. Add Student
2. Display All Students
3. Search Student by Name
4. Delete Student
5. Exit
Enter your choice (1-5): 2

---All Students---
No student records found.
```

10. Google Drive Submission Link

Google Drive folder containing all Python source files (.py):

 week_1_submission